

Handling Disappointment in C++

ISO/IEC JTC1 SC22 WG21 EWG P0157R0 - 2015-11-07

Lawrence Crowl, Lawrence@Crowl.org

[Introduction](#)

[Kinds of Disappointments](#)

[Traditional Approaches](#)

[Return Status](#)

[Intrusive Special Value](#)

[Status via Out Parameter](#)

[Return a Pair](#)

[Long Jump](#)

[Throw Exception](#)

[Analysis](#)

[Problem](#)

[Recent Approaches](#)

[Provide Two functions](#)

[Expected or Unexpected](#)

[Status and Optional Value](#)

[Comparison](#)

[Advisory Information](#)

[Efficiency of Return](#)

[Ease of Return](#)

[Recommendation to the Standard](#)

[Recommendation to Programmers](#)

Introduction

When a function fails to do what we want, we are disappointed. How do we report that disappointment to callers? How do we handle that disappointment in the caller?

In the discussion of a couple of new approaches to handling disappointment, the Evolution Working Group wanted general advice to programmers on how to answer those questions for their application. This paper provides that advice.

Kinds of Disappointments

There are many kinds of disappointments and programmers will want to report and handle them differently.

- The system implementation failed, e.g. out of memory.
- Something passed to the function failed, e.g. an exception thrown from a callback.
- The function detected an inappropriate environment, e.g. null pointer parameter.
- Data structure bounds prevent completion, e.g. pushing a full queue.
- The disappointment is expected, e.g. popping an empty queue.

Of these examples, the last two are not errors. Hence, we use the term disappointment instead of error.

Traditional Approaches

There are traditional C and C++ approaches to reporting and handling disappointments.

Return Status

The most common C approach is to return a status, typically as an int or enum, with success as a distinct value. There are a few problems with this approach.

- Handling the status obscures normal logic, which makes it hard to infer the normal logic.
- Writing the handler takes work that programmers would rather not do, which leads to ignoring any problems. Ignoring a bad status can quickly lead to undefined behavior.
- Returning a status displaces a natural return value to an 'out' parameter. These parameters require the caller to declare a

variable that the callee will overwrite.

- In practice, the an overwritable type must have a default constructor, or the functional equivalent. This restriction excludes some uses.
- For the caller to interpret and act on the status, the status must be known when the program is written, which means that status values not anticipated in the interface are problematic.

Intrusive Special Value

Instead of returning a status and displacing the natural return value, some C functions impose special disappointment semantics on one value of the return type. Typically, that value is a null pointer, zero, or negative one. There are problems.

- When that special is returned, the actual problem must be reported elsewhere.
- This report may be global object, like `errno`, which inhibits concurrency.
- When the problem is instead displaced to an 'out' parameter, the problems above occur.
- The known-in-advance problem occurs here as well.

Status via Out Parameter

Another common C-like approach is to have an 'out' parameter for the status. This approach has the benefit of not intruding on the natural return. However, it too has problems.

- It prevents placing the call in the condition to a loop or the head of a switch statement.
- While it is not possible to completely ignore the status, because a variable is required for the reference, you can easily not test it.
- In practice, the returned type must have a default constructor, or the functional equivalent, because there must be something to return when the function disappoints. This restriction excludes some uses.
- The known-in-advance problem occurs here as well.

Return a Pair

Another solution is to return a pair of status and value. In practice, these would then be tie'd to separate variables. While not yet common in C, this approach appears in other languages with built-in multiple return values. (See <http://blog.golang.org/error-handling-and-go> for the approach in the Go programming language.)

This approach has essentially the same attributes as the approaches above. The primary difference is that one need to declare a separate variable to hold the 'other' return object.

Long Jump

Some applications use long jump to handle disappointments. The problem is that long jump has no mechanism to clean up state in intermediate frames. Consequently, it is usable only in very constrained situations where either there is no state in intermediate frames or the program can abandon that state. Given this constraint, we do not consider it further.

Throw Exception

The C++ exception mechanism addresses the problems above.

- Exception handling code is separate from normal code, so the normal logic is not obscured.
- The return type need not have an effective default constructor.
- Exceptions that are not explicitly handled are propagated up the call stack.
- The exception mechanism handles unknown disappointments as well as known disappointments.
- Exceptions propagating through a stack frame will properly destroy local objects, enabling programming-free handling for many local objects.

Unfortunately, the exception mechanism introduced other problems.

- The exception mechanism is expensive. Reporting common disappointments via exceptions can consume a significant amount of processor time.
- The exception mechanism induces a large variance in the cost of a function call. This variance could cause problems on systems with hard real-time constraints.
- Separation of the exception handling code from normal code can obscure the logic when the action required for a disappointment is to adjust and try again. For example, reporting that a hash table does not have an entry via an exception would take substantially more code than a boolean return.

Analysis

We can group traditional approaches into two broad categories by examining their attributes.

attribute	status-based	exception-based
normal logic is clearer when disappointments are normally	addressed and redone	passed on to other code
the effect of ignoring disappointment is	often undefined behavior	local variable destruction and exception propagation
disappointments are applicable when	known in advance	not known in advance
some form of default construction is	required	not required
handling overhead is inefficient when disappointments are	rare	not rare
accomodating real-time constraints is	easier	harder

The first three attributes are variations on *actionable*. A corrupt file system is rare and unlikely to be actionable in the caller. On the other hand, an empty queue is likely common and likely actionable.

In summary, the status-based approach is best when disappointments are actionable and not rare or when there are hard real-time constraints, while the exception-based approach is best when disappointments are not actionable and rare.

Problem

The problem with traditional approaches is that whether or not a disappointment is actionable or rare may depend on the calling environment, but the implementation of the function does not. Whether the environment has real-time constraints may not be known to the programmer of the function.

As an example, a function reading a system file can expect to find it present, while a function looking for a user's dot file can expect to find it not present. A more program-internal example is an application that knows it will not fill a queue, and so a full queue indicates a rare error. On the other hand, another application may rely on a full queue to provide flow control.

Programs will be clearer, more efficient and more robust when we can leave at least some of the choice in mechananism to the caller.

Recent Approaches

There have been several new approaches to handling disappointment developed and deployed recently. All these approaches address the primary problem of the character of the disappointment being known only in the calling environment.

Provide Two functions

The first solution is to provide two versions of each function, one providing a status and one throwing an exception. This approach enables choice of mechanism at each call site.

- In [N3533 C++ Concurrent Queues](#), the non-throwing function returns a status.
- In [N4100 Programming Languages — C++ — File System Technical Specification](#), the non-throwing function writes the status through a reference parameter. [N2838 Library Support for Hybrid Error Handling](#) proposed a change to standard library specifications so that

```
void f(error_code& ec=throws());
```

would stand both overloads of the function, the one without the reference parameter throwing and the one with the reference parameter not throwing.

The non-throwing function in this approach shares the problem of effectively requiring a default constructor with the traditional pure-status approach.

While less likely in practice, with two functions it is still possible to request a status, ignore it and access a missing result. Whether or not this access is undefined behavior depends on whether or not the default constructor produces an object with defined behavior. In any case, the default object is unlikely to produce what one wants.

Expected or Unexpected

[N4109 A proposal to add a utility class to represent expected monad](#) proposes a class template `expected` to contain either the normal return object or an exceptional object, but not both.

A conversion from `expected` to `bool` enables determining if the expected value is present. The dereference operator returns an *unchecked* reference to the expected value. The `value` member function returns a checked reference to the expected value.

Accessing a missing result is possible by using the dereference operator. The behavior is undefined if one does so.

Status and Optional Value

[N4233 A Class for Status and Optional Value](#) proposes a class template `status_value` to contain a status and possibly the normal return object.

A conversion from `status_value` to `bool` enables determining if the value is present. The dereference operator returns an *checked* reference to the expected value. The `value` member function returns a checked reference to the expected value.

Accessing a missing result is not possible.

Comparison

We compare the recent approaches on three points.

Advisory Information

The `status_value` proposal differs from the `expected` proposal in that it always provides a status. By always having a status that status can provide advisory information in addition to the normal return value. It can say, in effect, "I was able to satisfy your need this time, but in the future you need to modify your behavior to reduce the risk of future disappointment". A couple of examples are in order.

- A hash table insertion returns a status "success but the table is getting full". This allows the calling environment to choose an appropriate time for cleaning or growing the hash table.
- A concurrent data structure return a status "success but under contention". This allows the calling environment to defer or ignore non-essential work to keep response time low.

The two-function approach can also provide advisory information in the case where one chooses the status-based function.

With all three approaches, it is possible to ignore advisory information. In the two-function approach, simply use the exception-throwing version. In the other two approaches, use the conversion-to-bool for decisions.

Efficiency of Return

The three approaches have roughly the same efficiency when the exceptional object is cheap, on the order of an enum. As exceptional objects become more expensive, the status-based function and the `status_value` type pay an increasing cost. The exception-based function and the `expected` type avoid paying that cost for the non-exceptional case.

This difference in efficiency shows most clearly in the file system technical specification. The specification uses the two-function approach. The status-based functions return a `error_code`, while the exception-based functions throw a `filesystem_error` that contains an `error_code` plus additional diagnostic information. The `status_value` type, as is, encourages simple enumerations for status. Returning a full `filesystem_error` on every call would be relatively expensive. In contrast, the `expected` type would only construct a `filesystem_error` at need.

Ease of Return

While not a major concern, the status is required in construction of a `status_value` object is simpler than construction of an `expected` object.

While use of the two function approach would seem simpler than either of the above, it would likely not prove so in practice. To prevent redundant implementation, the main work is likely to be done in the status-based function and the other exception-based function acts as a wrapper. Thus, the code is likely more complex for the two-function approach.

Recommendation to the Standard

Both `expected` and `status_value` provide value and should be considered for adoption. Some determine effort to unify the two proposal would likely result in a more consistent outcome.

There are some technical changes to the proposals that would improve them.

- Change the `status_value::status` operation to return a reference rather than a value. This change enables more expensive status types.
- Change the `expected` dereference operator to check for a valid expected object. While the lack of a check is present for efficiency, that efficiency would likely be gained by an optimizing compiler recognizing a redundant comparison when the `expected` functions are inlined.
- Consider change the `expected` construction to have a single constructor with a discriminator parameter rather than two auxiliary maker functions.

Recommendation to Programmers

Programmers of a function should consider how to communicate disappointment to their callers with the following advice, taken in order.

- If the function will never disappoint, use `noexcept` and do not have a status.
- If there are significant real-time constraints on the program, where the cost of an exception is prohibitive, use a status-based function. In such programs, a strong review program will be necessary to ensure that all disappointments are handled. This path will prohibit some types and programs.
- If a disappointment not known in advance, throw an exception. Programmers cannot handle what they cannot know. The most common instance of this case is a function invoking a callback function that throws. The callback function may throw exceptions unknown to the outer function.
- If a disappointment is rare and not actionable, throw an exception. The concurrent queue returns a status from most functions, but no status includes a mutex failure. When a mutex fails, the queue operations will throw an exception.
- If the function may provide advisory information, use `status_value`. Callers can decide whether a disappointment is rare or actionable.
- If the exceptional object is expensive to construct, use `expected`. Callers can decide whether a disappointment is rare or actionable. (The exceptional object need/should not have a 'success' value, as the programmer has access only to the exceptional object on success.)
- Otherwise, use `status_value` or `expected` based on which seems more natural for code explicitly handling disappointment.